

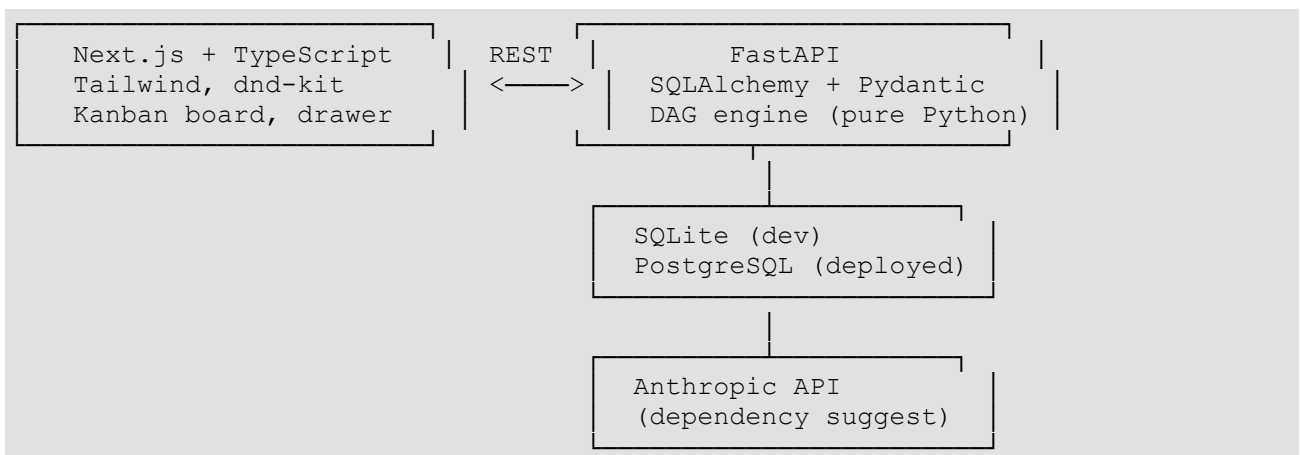
TaskFlow Pro — Architecture, Data Model & Known Limitations

1. What this document covers

This is the combined design write-up required for the Build submission: how the system is put together, how the data is modeled, and where it currently falls short. We wrote it after most of the engine and API work was done, so it reflects what we actually built rather than what we planned at the start — a few things changed along the way, and we've noted where.

2. High-level architecture

TaskFlow Pro is split into two halves that don't trust each other: a Next.js frontend that people interact with, and a FastAPI backend that owns every decision about the task graph. We went back and forth early on about whether to compute Blocked/Ready status in the browser for a snappier feel, but decided against it — if the client and server ever disagreed about which tasks were blocked, that's a much worse bug than a slightly slower UI. So the browser only ever displays what the server tells it, and drag-and-drop moves are optimistic on screen but reverted if the server rejects them.



2.1 Frontend

The board is a standard four-column Kanban (Backlog, In Progress, Review, Done) built with dnd-kit for drag-and-drop. Each card shows its Blocked/Ready badge, its dates, and how many prerequisites it has. Clicking a card opens a drawer where you can edit the task and manage its dependencies — adding a link runs it through the server's cycle check, and if it fails, the drawer shows the exact cycle (e.g. "A → B → C → A") instead of just a generic error, which we found much more useful when debugging our own seed data.

There's also a "schedule impact" panel that appears after any change that moves dates around. It lists which tasks shifted and by how much, so the person making the change can actually see the ripple effect instead of having to click into every downstream card.

2.2 Backend

FastAPI handles routing, validation and persistence. The actual scheduling logic lives in its own module with no FastAPI or SQLAlchemy imports at all — we kept it as plain Python operating on simple in-memory structures (adjacency lists, in-degree counts) so it could be unit tested in isolation before the API or the UI existed. This turned out to be the right call: most of our early bugs (see section 4) were caught by engine tests alone, hours before we'd written a single React component.

The engine is responsible for:

- rejecting cycles before anything is written to the database
- deriving each task's Blocked/Ready state from its prerequisites' status
- recomputing dates for downstream tasks after any change, once per task, using topological order
- finding the critical path (the longest chain by total duration)

Every mutating endpoint wraps its work in a single database transaction, so a rejected dependency or an invalid move never leaves the graph half-updated.

2.3 AI / LLM layer

The Anthropic API is called only from the backend, on a specific endpoint the user triggers manually (there's a "Suggest dependencies" button — nothing happens automatically). We send task titles and descriptions, ask for a strict JSON response, and then run every suggestion the model gives us back through our own validation before it's even shown to the user: unknown task ids are dropped, suggestions with a quote that doesn't actually appear in the task text are dropped, duplicates are dropped, and anything that would create a cycle is dropped using the same check we use for manual edits. What's left is stored as "pending" and shown with its reasoning; a human has to click Accept before it becomes a real dependency. We treat this as non-negotiable — the model proposes, the engine and the user decide.

3. Data model

Three tables carry the whole app.

tasks | Column | Type | Notes | |---|---|---| | id | UUID / int | primary key | | title | text | | | description | text | used as context for AI suggestions | | status | enum | Backlog / In Progress / Review / Done | | position | int | ordering within a column | | duration_days | int | | | constraint_start | date | earliest date the user allows the task to start | | start_date | date | derived by the engine | | end_date | date | derived by the engine | | created_at / updated_at | timestamp | |

dependencies | Column | Type | Notes | |---|---|---| | prerequisite_id | FK → tasks.id | | | dependent_id | FK → tasks.id | |

Unique constraint on the (prerequisite_id, dependent_id) pair, a check constraint so a task can't depend on itself, and ON DELETE CASCADE so removing a task cleans up its edges.

dependency_suggestions | Column | Type | Notes | |---|---|---| | id | int | | | from_id / to_id | FK → tasks.id | proposed prerequisite / dependent | | confidence | float | 0–1, from the model | |

rationale | text | model's explanation | | evidence | text | quote from task text, verified server-side | | state | enum | pending / accepted / rejected | | model | text | which model produced it | | created_at | timestamp | |

One thing worth calling out: **Blocked and Ready are not columns anywhere.** We deliberately don't store them, because a stored flag can drift out of sync with reality the moment a prerequisite's status changes elsewhere. It's cheap enough to compute on read that persisting it wasn't worth the risk of it going stale.

4. Known limitations

We'd rather be upfront about these than have them discovered during review.

- **Finish-to-start only.** Every dependency means "the prerequisite must fully finish before the dependent can start." We don't support start-to-start, finish-to-finish, or lag/lead time between tasks. For a hackathon scope this covers the vast majority of real project dependencies, but it's a simplification.
- **Calendar days, not working days.** Duration and scheduling don't account for weekends or holidays. A 3-day task starting Friday finishes Monday in our model, which isn't how most teams actually plan.
- **Single workspace, effectively single user.** There's no multi-tenant separation and no real-time collaboration — if two people edit the same graph at once, the second write wins. We treat this as acceptable for the demo but it would need proper concurrency handling (locking or CRDT-style merging) before it could be used by an actual team.
- **No resource or capacity constraints.** The engine only reasons about ordering and dates, not about whether a person or team is actually available to do the work. Two tasks can be "Ready" and scheduled on overlapping dates even if they'd require the same person.
- **A Blocked task can't be dragged forward.** We chose to hard-block the move (with an explanation of which prerequisite isn't done) rather than allow it with a warning. This wasn't spelled out in the problem statement, so it's a judgment call — a stricter interpretation of "the engine enforces correctness of the graph," but arguably less flexible for real usage where people sometimes need to jump the queue.
- **AI suggestions only look at title and description.** They don't consider board position, due dates, or the existing graph structure beyond avoiding cycles and duplicates. A more sophisticated version could weigh timing or team ownership, but that felt out of scope for a 72-hour build.
- **No offline support.** The frontend assumes a live connection to the backend; there's no local caching or optimistic queueing beyond a single in-flight drag action.
- **Critical path recalculates in full each time.** It's not incrementally maintained, so on a very large graph (thousands of tasks) it would be the first thing worth optimizing.

None of these are things we ran out of time for blindly — they were conscious calls about where to spend 72 hours, and we'd rather list them here than have them look like oversights.